GLM - Part II

Aliaksandr Hubin

April 21 2026

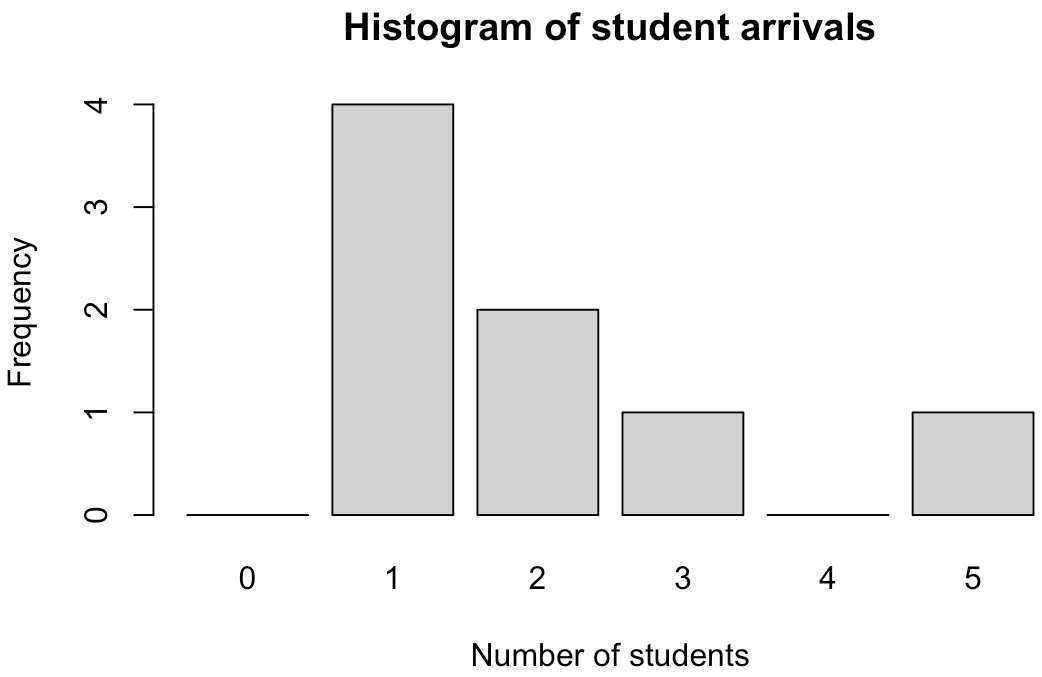
Today's topics

- Count data
- Example and theory
- Overdispersion and quasi-likelihood
- Gamma regression
- Multinomial regression

Example - Student arrivals

Number of students coming in the last minute before the lecture so far in the course:

```
students <- c(2, 1, 1, 2, 1, 5, 3, 1)
```



Count response

$$y_i \in [0, 1, \dots \infty)$$

for $i = 1, \dots, n$

We assume

$$y_i \sim Poisson(\lambda_i)$$

Further, theory says:

$$\mu_i = E(y_i) = \lambda_i > 0$$

$$\sigma_i^2 = Var(y_i) = \lambda_i > 0$$

Poisson probability model

The probability mass function mapping the probability of exactly k events is given by:

$$P(y_i = k) = \frac{\lambda_i^k \exp(-\lambda_i)}{k!}$$

Estimating the Poisson parameter

For the student arrivals data, we can estimate the parameter λ using the maximum likelihood estimate (MLE), which is the sample mean.

```
# MLE of Poisson parameter lambda is the sample mean
lambda_hat <- mean(students)
lambda_hat
```

```
## [1] 2
```

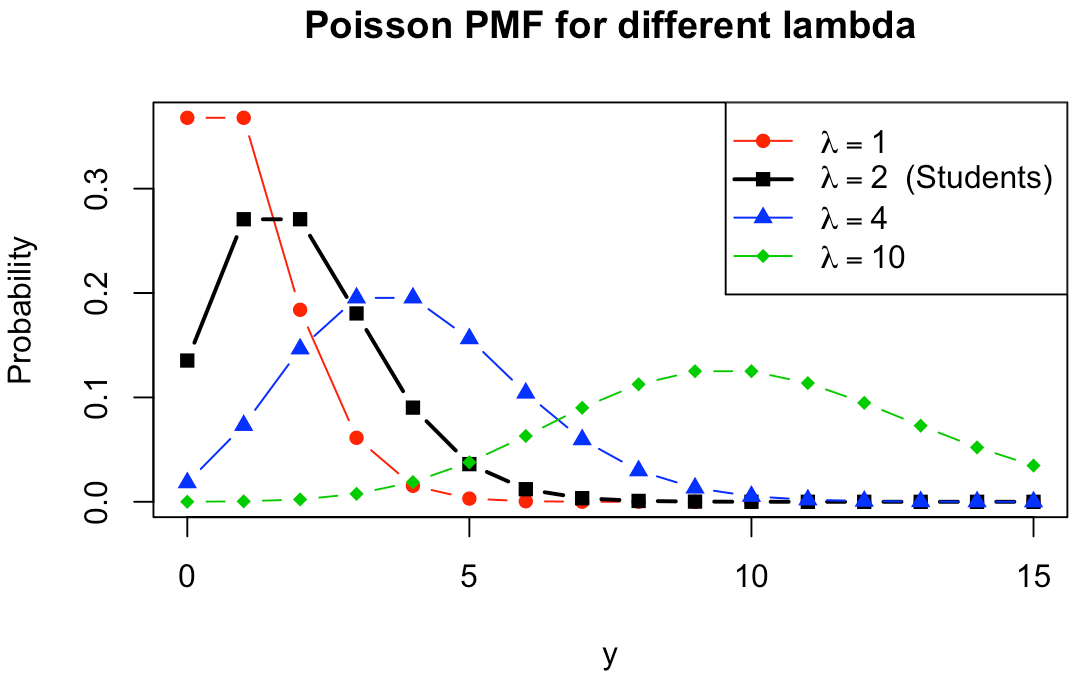
So for students $\lambda_i = 2$:

- No-one late ($k = 0$): $P(y_i = 0) = \frac{2^0 \exp(-2)}{0!} = e^{-2} \approx 0.135$

- Two late ($k = 2$): $P(y_i = 2) = \frac{2^2 \exp(-2)}{2!} = 2e^{-2} \approx 0.271$

Shapes of the Poisson distribution

The shape of the Poisson PMF changes directly with λ. Here we also overlay $\lambda = 2$ representing our student arrivals data.



The log-link function

The canonical link function for Poisson distributed response is the `log`-link.

Hence we assume that the linear predictor

$\eta_i = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}$ is related to the mean through the link function:

$$g(\lambda_i) = \log(\lambda_i)$$

by:

$$\log(\lambda_i) = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}$$

Since we want:

$$\lambda_i = \exp(\beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}) > 0$$

The inverse link-function

The inverse link function $\exp()$ provides the estimates of the mean from estimated regression coefficients:

$$\hat{\lambda}_i = \exp(\hat{\beta}_0 + \hat{\beta}_1 x_{1i} + \cdots + \hat{\beta}_p x_{pi})$$

Example - Plant Species

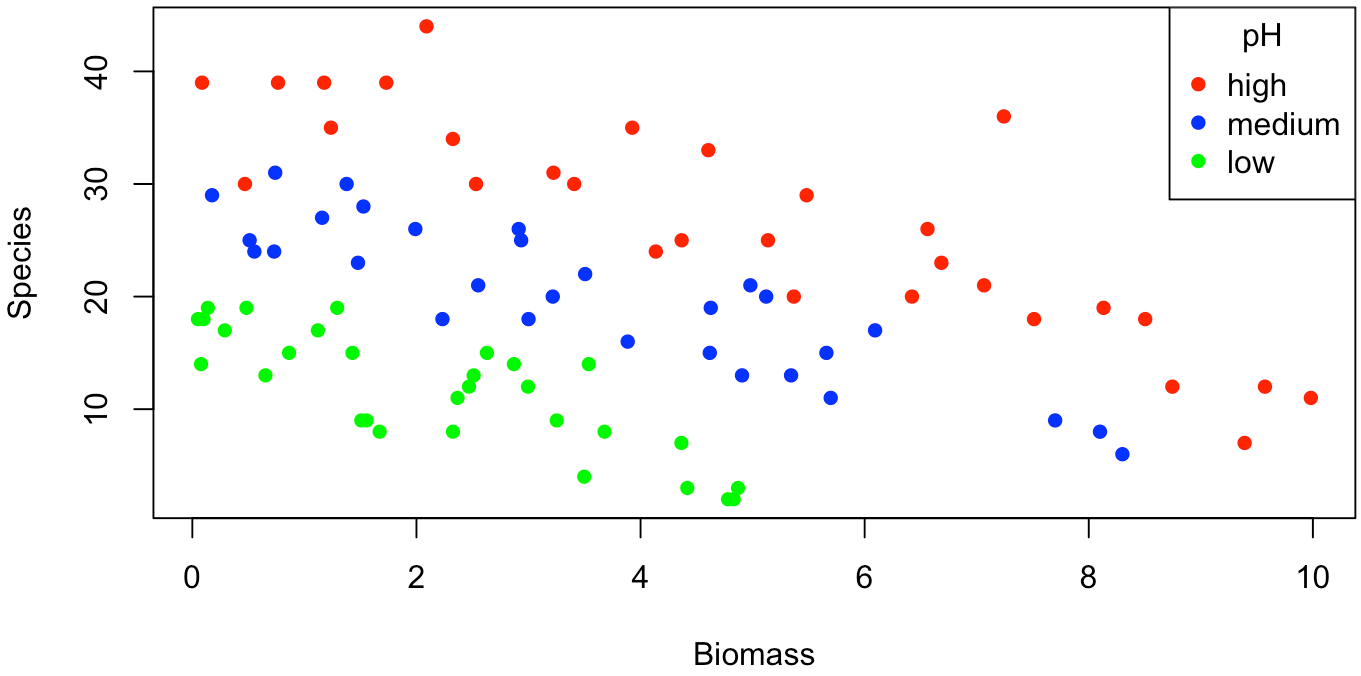
Number of plant species in plots of different biomass and pH.

```
library(BIAS.data)
data(species)
head(species, 4)
```

```
##     pH   Biomass Species
## 1 high 0.4692972      30
## 2 high 1.7308704      39
## 3 high 2.0897785      44
## 4 high 3.9257871      35
```

Example - Plant Species (Cont.)

Number of plant species in plots of different biomass and pH.



Species count as function of biomass

A Poisson regression with `Biomass` as predictor:

```
mod1.glm <- glm(Species ~ Biomass, family=poisson(log), data=species)
summary(mod1.glm)
```

Output

A Poisson regression with `Biomass` as predictor:

```
##
## Call:
## glm(formula = Species ~ Biomass, family = poisson(log), data = species)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  3.184094   0.039159   81.31  < 2e-16 ***
## Biomass     -0.064441   0.009838   -6.55 5.74e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 452.35  on 89  degrees of freedom
## Residual deviance: 407.67  on 88  degrees of freedom
## AIC: 830.86
##
## Number of Fisher Scoring iterations: 4
```

Deviance test for the effect of biomass

```
library(mixlm)
anova(mod1.glm,test='LR')
```

Output

```
## Analysis of Deviance Table
##
## Model: poisson, link: log
##
## Response: Species
##
## Terms added sequentially (first to last)
##
##
##         Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
## NULL                       89     452.35
## Biomass  1   44.673        88     407.67 2.328e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Conclusion: Biomass is highly significant with a negative effect on species count.

Delta unit interpretation

Using a log-link makes the interpretation of the slope **multiplicative**. This is easy to derive:

$$\log(\mu_{\text{new}}) - \log(\mu_{\text{old}}) = (\beta_0 + \beta_1(x+1)) - (\beta_0 + \beta_1 x) = \beta_1$$

$$\implies \log\left(\frac{\mu_{\text{new}}}{\mu_{\text{old}}}\right) = \beta_1 \implies \mu_{\text{new}} = \mu_{\text{old}} \exp(\beta_1)$$

Taking a 1-unit delta (increase) in predictor x_j **multiplies** the expected **mean** response **by** $\exp(\beta_j)$.

For example, $\hat{\beta}_{\text{Biomass}} = -0.0644$. A 1-unit increase in `Biomass` multiplies the expected species count by $\exp(-0.0644) \approx 0.938$, which corresponds to a decrease of 6.2%.

Prediction

What is the predicted species count for a plot with biomass equal to 4?

```
new <- data.frame(Biomass=4)
    link <- predict(mod1.glm, new, type = "link", se.fit = TRUE)
    response <- predict(mod1.glm, new, type = "response", se.fit = TRUE)
    LS <- cbind(link$fit, link$se.fit, response$fit, response$se.fit)
    colnames(LS) <- c("link", "se.link", "response", "se.response")
    LS
```

Output

```
##       link    se.link response se.response
## 1 2.926332 0.02530713 18.65906   0.4722072
```

This was found by letting $x^* = 4$ and

$$\hat{\eta}^* = 3.184 - 0.0644 \cdot x^* = 2.926$$

$$\hat{y}^* = \hat{\lambda}^* = \exp(\hat{\eta}^*) = \exp(2.926) = 18.66$$

Expanding the model

From the plot we saw that pH may have large influence.

```
mod2.glm <- glm(Species ~ Biomass + pH, family=poisson, data=species)
summary(mod2.glm)
```

Output (Expanded model)

From the plot we saw that pH may have large influence.

```
##
## Call:
## glm(formula = Species ~ Biomass + pH, family = poisson, data = species)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  3.32176    0.03883   85.54   <2e-16 ***
## Biomass     -0.12756    0.01014  -12.58   <2e-16 ***
## pH(high)     0.52718    0.03400   15.50   <2e-16 ***
## pH(low)     -0.60921    0.04125  -14.77   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 452.346  on 89  degrees of freedom
## Residual deviance:  99.242  on 86  degrees of freedom
## AIC: 526.43
##
## Number of Fisher Scoring iterations: 4
```

Deviance tests

```
anova(mod2.glm,test='LR')
```

```
## Analysis of Deviance Table
##
## Model: poisson, link: log
##
## Response: Species
##
## Terms added sequentially (first to last)
##
##
##          Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
## NULL                        89     452.35
## Biomass  1   44.673        88     407.67 2.328e-11 ***
## pH       2  308.431        86      99.24 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Over/under-dispersion

$Var(y)$ depends on the mean μ in specific ways for the binomial and the Poisson distribution (and others).

Sometimes data vary more (over-dispersion) or less (under-dispersion) than expected.

Over-dispersion is very common in biological data

Usually this is due to some (unknown) factor influencing the data.

The quasi-likelihood

The quasi-likelihood method relaxes the variance assumption by including a `scale`-parameter.

Over-dispersion corresponds to `scale > 1`

Under-dispersion corresponds to `scale < 1`

We can estimate the scale-parameter using the `family = quasi–` distributions

If pH was "unknown"…

Assume `pH` was not measured (back to the first model…)

Let's fit a `quasi-poisson` model and look at the estimates. No change!

```
mod1q.glm <- glm(Species ~ Biomass, family=quasipoisson, data=species)
sumfit <- summary(mod1q.glm)
sumfit$coefficients
```

Quasi-Poisson Output

```
##
## Call:
## glm(formula = Species ~ Biomass, family = quasipoisson, data = species)
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.18409    0.08192  38.870  < 2e-16 ***
## Biomass     -0.06444    0.02058  -3.131  0.00236 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 4.375969)
##
##     Null deviance: 452.35  on 89  degrees of freedom
## Residual deviance: 407.67  on 88  degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 4
```

Compare with the model assuming poisson distribution:

```
##                Estimate  Std. Error   z value     Pr(>|z|)
## (Intercept)  3.18409377 0.039159103 81.311714 0.000000e+00
## Biomass     -0.06444054 0.009837528 -6.550481 5.735197e-11
```

But note the increased standard errors!

Estimated over/under-dispersion

The estimated dispersion parameter is:

```
summary(mod1q.glm)$dispersion
```

```
## [1] 4.375969
```

The standard errors of the estimated regression coefficients have increased by a factor of $\sqrt{4.376} = 2.091$

There exist formal tests for over/under-dispersion:

- Parametric Pearson residuals test
- Nonparametric Pearson residuals test
- Simulation-based residual variance test
- Etc.

Gamma regression

For continuous variables that are strictly positive and often right-skewed, we can use the **Gamma distribution**.

Its probability density function is:

$$f(y) = \frac{1}{\Gamma(\nu)} \left(\frac{\nu}{\mu} \right)^{\nu} y^{\nu-1} \exp\left(-\frac{\nu y}{\mu} \right)$$

for $y > 0$, where $\mu > 0$ is the mean and $\nu > 0$ is the shape parameter.

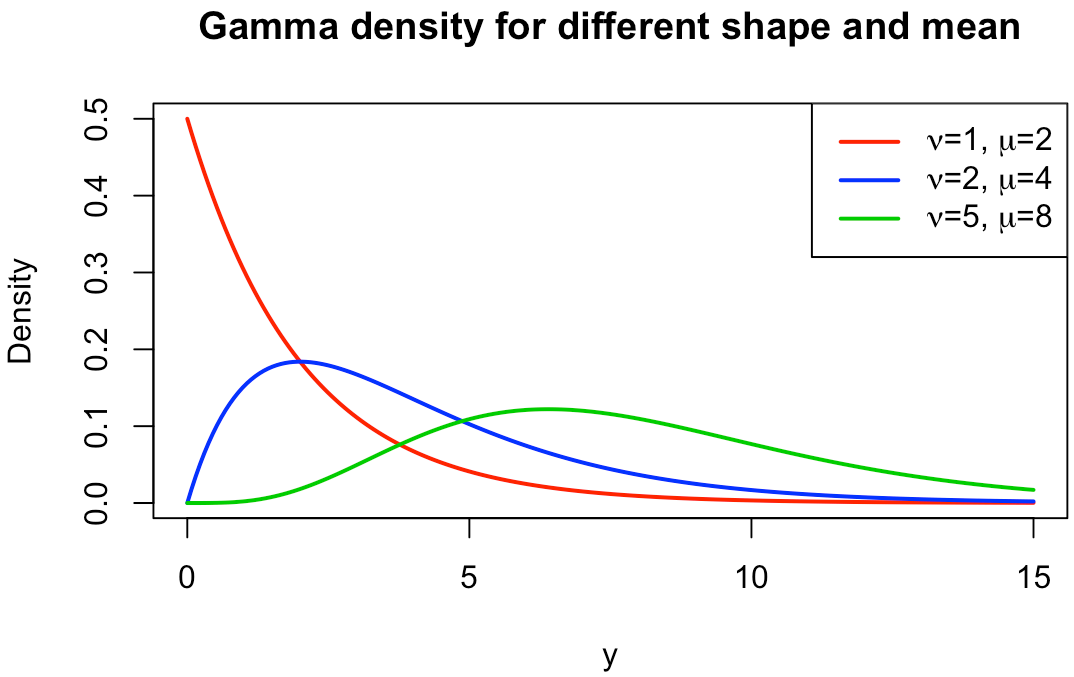
The variance of a Gamma distributed variable is proportional to the square of its mean, $Var(y) = \mu^2/\nu$, making it ideal when larger responses have higher variance.

To ensure that the predicted mean μ is always positive, we typically use the **log-link**:

$$g(\mu) = \log(\mu) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Shapes of the Gamma distribution

The Gamma PDF is highly flexible based on the shape (ν) and mean (μ).



Gamma regression example

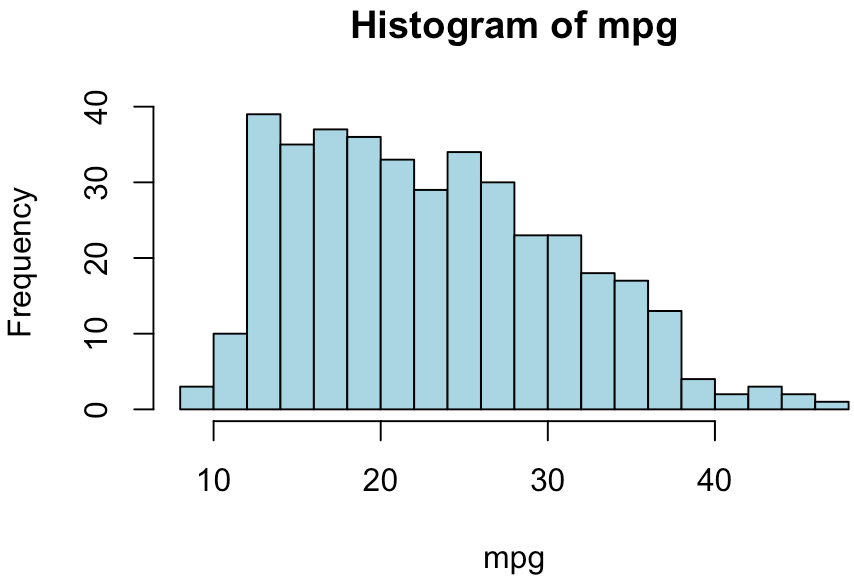
Consider the `Auto` dataset from the `ISLR` package. We want to predict the continuous and strictly positive `mpg` (miles per gallon) based on vehicle `weight` and `horsepower`.

```
library(ISLR)
data(Auto)
head(Auto[, c("mpg", "weight", "horsepower")], 3)
```

```
##   mpg weight horsepower
## 1  18   3504        130
## 2  15   3693        165
## 3  18   3436        150
```

Why not Gaussian?

If we look at the distribution of `mpg`, it is strictly positive and right-skewed. A standard Gaussian regression assumes symmetric errors with constant variance, which is not suitable here.



Gamma regression fit

```
mod.gamma <- glm(mpg ~ weight + horsepower,
                 family=Gamma(link="log"), data=Auto)
```

```
##
## Call:
## glm(formula = mpg ~ weight + horsepower, family = Gamma(link = "log"),
##     data = Auto)
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.128e+00  2.991e-02 137.994  < 2e-16 ***
## weight      -2.504e-04  1.895e-05 -13.218  < 2e-16 ***
## horsepower  -2.602e-03  4.181e-04  -6.224 1.26e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Gamma family taken to be 0.02557305)
##
##     Null deviance: 44.2046  on 391  degrees of freedom
## Residual deviance:  9.6637  on 389  degrees of freedom
## AIC: 2099.6
##
## Number of Fisher Scoring iterations: 4
```

Gamma Interpretation

Both predictors have a significant negative effect on the expected `mpg`. The variance assumption of the Gamma model naturally handles the heteroscedasticity common in this type of data.

Because we use the **log-link**, a 1-unit increase in the predictor x_j multiplies the expected mean response by $\exp(\beta_j)$. For instance, since $\hat{\beta}_{\text{weight}} = -0.00025$, an increase of 1000 units in `weight` multiplies the expected `mpg` by $\exp(-0.25) \approx 0.78$.

The Multinomial Distribution

Consider N **independent trials**, each resulting in exactly one of t **categories** (bins) with fixed probabilities $\mathbf{p} = (p_1, p_2, \ldots, p_t)$, where $\sum_{j=1}^{t} p_j = 1$.

Let Y_j = number of trials falling in category j. Then the vector $(Y_1, Y_2, \ldots, Y_t)$ follows a **Multinomial distribution**:

$$\mathbf{Y} \sim \text{Multinomial}(N, \mathbf{p})$$

with joint probability mass function:

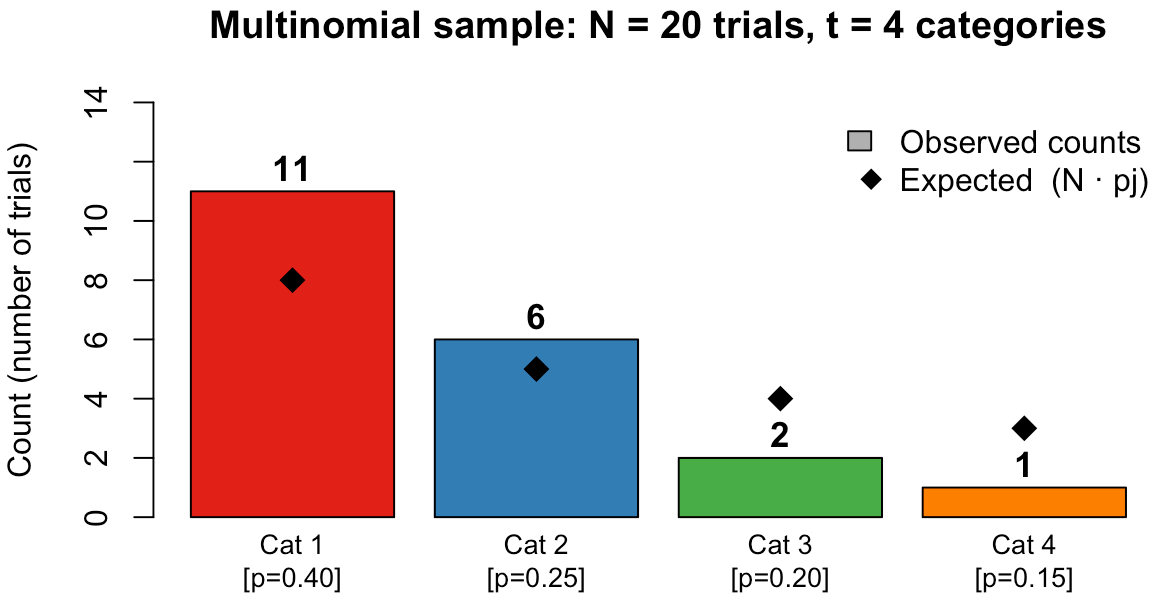
$$P(Y_1 = k_1, \ldots, Y_t = k_t) = \frac{n!}{k_1!\, k_2!\, \cdots\, k_t!}\; p_1^{k_1}\; p_2^{k_2}\; \cdots\; p_t^{k_t}$$

where $k_1 + k_2 + \cdots + k_t = N$ and for each j: $E(Y_j) = np_j$,
$Var(Y_j) = Np_j(1 - p_j)$.

Note: When $t = 2$ this reduces to the familiar **Binomial** distribution.

Multinomial: Bins & Trials Illustration

$n = 20$ trials distributed across $t = 4$ categories:



Each bar = one **bin** (category); bar height = how many trials landed there.

Categorical Distribution when $N = 1$

When we perform only **one trial** ($N = 1$), each $Y_j \in \{0, 1\}$ and exactly one $Y_j = 1$. This is the **Categorical distribution**:

$$Y \sim \text{Categorical}(\mathbf{p}), \quad P(Y_j = 1) = p_j, \quad j = 1, \dots, t$$

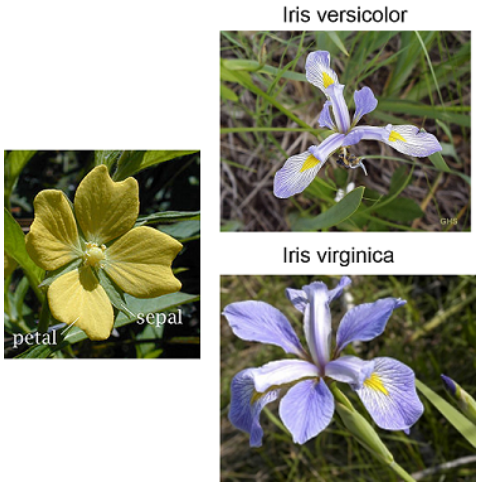
Think of it as a single **roll of a weighted** t-sided die.

Connection to regression: In multinomial regression each observation i is modelled as one Categorical trial with class-probabilities p_{ij} driven by covariates $\mathbf{x}_i$.

Example - Iris Species

We want to predict `Species` (setosa, versicolor, virginica) of iris flowers based on their `Sepal.Length`.

Let i denote a flower, and we have categorical outcome $j = 1$ (setosa), $j = 2$ (versicolor), $j = 3$ (virginica).




```
##   Sepal.Length Species
## 1          5.1  setosa
```

Multinomial model assumptions

Given a flower-specific linear predictor η_{ij} for flower i and species j, the probability of outcome j is modelled as:

$$p_{ij} = \frac{\exp(\eta_{ij})}{\sum_k \exp(\eta_{ik})}; \text{known as softmax function}$$

where $\eta_{ij} = \beta_{0j} + \beta_{1j} \cdot \text{Sepal.Length}_i + \beta_{2j} \cdot \text{Sepal.Width}_i$.

We use baseline response parameterization (default in R) where the first category (setosa) is the reference, so we set $\eta_{i1} = 0$. This gives a simplification for $j = 1$:

$$p_{i1} = 1 - p_{i2} - p_{i3}$$

R code (Baseline parameterization)

Let's use the `nnet` package to fit the model.

```
MLM.iris <- nnet::multinom(Species ~ Sepal.Length + Sepal.Width,
                           data=iris, trace=FALSE)
summary(MLM.iris)
```

```
## Call:
## nnet::multinom(formula = Species ~ Sepal.Length + Sepal.Width,
##     data = iris, trace = FALSE)
##
## Coefficients:
##            (Intercept) Sepal.Length Sepal.Width
## versicolor   -92.09924     40.40326   -40.58755
## virginica   -105.10096     42.30094   -40.18799
##
## Std. Errors:
##            (Intercept) Sepal.Length Sepal.Width
## versicolor    26.27831     9.142717    27.77772
## virginica     26.37025     9.131119    27.78875
##
## Residual Deviance: 110.425
## AIC: 122.425
```

Here `setosa` is the baseline response. The model estimates two sets of coefficients: one for `versicolor` vs `setosa` and one for `virginica` vs `setosa`.

Calculating Probabilities

For a given flower with `Sepal.Length = 6` cm and `Sepal.Width = 3` cm, the linear predictors are computed from the estimated coefficients (Remember $\widehat{\eta}_1 = 0$):

```
new_data <- data.frame(Sepal.Length=6, Sepal.Width=3)
round(predict(MLM.iris, newdata=new_data, type="probs"), 4)
```

```
##     setosa versicolor  virginica
##     0.0000     0.6028     0.3972
```

Classification Accuracy (Linear Model)

We can evaluate the model's classification accuracy over the entire dataset:

```
predicted_classes <- predict(MLM.iris, newdata=iris, type="class")
accuracy <- mean(predicted_classes == iris$Species)
round(accuracy, 3)
```

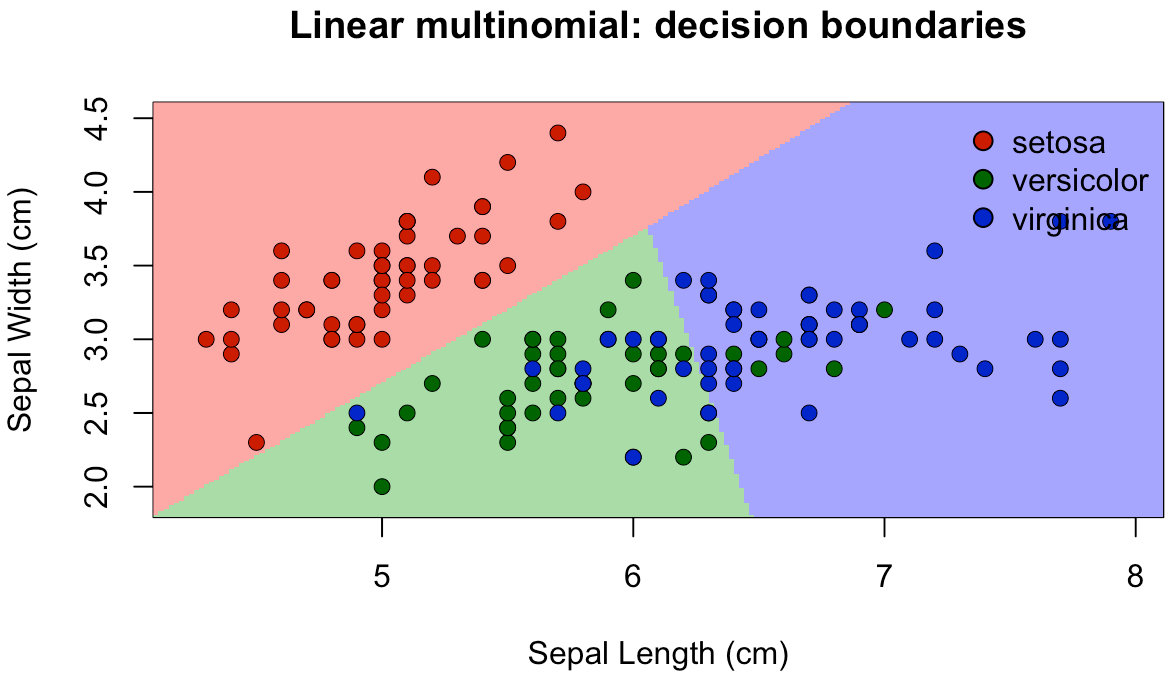
```
## [1] 0.833
```

```
table(Predicted = predicted_classes, Actual = iris$Species)
```

```
##             Actual
## Predicted    setosa versicolor virginica
##   setosa         50          0         0
##   versicolor      0         38        13
##   virginica       0         12        37
```

Linear Decision Boundary in 2D

Another model: `Species ~ Sepal.Length + Sepal.Width`.
Background colours show the predicted class region; points are true species.



Cubic Decision Boundary in 2D

Model: `Species ~ poly(Sepal.Length, 3) + poly(Sepal.Width, 3)`.
Polynomial terms allow curved, non-linear class boundaries.